Voxtral Real-Time 4B Compression — Complete Battle Plan

Resilient AI Challenge | Track 2: Audio-to-Text

Your hardware: RTX 5080 (16GB VRAM) + 32GB system RAM **Deadline Round 1:** April 27, 2026 (submissions open) → May 4 (closes) **Deadline Round 2:** May 20 → June 15, 2026 (final) **Eval hardware:** NVIDIA L4 (24GB VRAM) on Aguinic Phase platform **Winning metric:** Lowest energy consumption (joules) while maintaining ≥80% of baseline WER **Inference engine:** vLLM ONLY (no llama.cpp — Voxtral's architecture is only supported in vLLM)

PART 1: UNDERSTAND WHAT YOU'RE COMPRESSING

1.1 Model Architecture

Voxtral Real-Time is NOT a simple LLM. It's a multimodal streaming ASR model with three distinct components:

```
Audio (16kHz mono)
  → Mel Spectrogram [B, 128, T]
  → Causal Audio Encoder (32 layers, 1280 dim, ~0.6B params)
  → Conv 4x downsample
  → Reshape [B, T/16, 5120]
  → MLP Adapter [B, T/16, 3072] (~small, <0.1B params)
  → Autoregressive Text Decoder (26 layers, 3072 dim, ~3.4B params)
  → Token IDs → Text
```

Component	Parameters	Layers	Dim	Details
Audio Encoder	~0.6B	32	1280	Causal transformer, sliding window 750 frames (15s)
Adapter	~0.05B	2 linear	5120→3072	MLP with GELU, 4x temporal downsampling
Text Decoder	~3.4B	26	3072	Ministral-3B backbone, GQA (32 Q heads, 8 KV heads)

Total	**~4B**	—	—	Apache 2.0 license

Critical architecture details:

- Decoder uses GQA with 32 query heads and 8 KV heads
- Decoder sliding window attention: 8192 tokens
- Encoder sliding window: 750 frames (15 seconds of audio)
- Audio and text embeddings are SUMMED (not concatenated) at each position
- Ada-RMSNorm conditioning: delay parameter τ modulates decoder via sin/cos embedding
- Special tokens: [P] (padding/wait), [W] (word boundary), [STREAMING_PAD]
- Frame rate after adapter: 12.5 Hz (each frame = 80ms of audio)
- Default max-model-len: 131072 tokens (~3 hours of audio)

Why this architecture matters for compression:

- The encoder (0.6B) is EXTREMELY sensitive to quantization — it processes raw audio features and any precision loss directly impacts WER
- The decoder (3.4B, 85% of params) is a standard LLM backbone and is much more robust to quantization
- The adapter is tiny — leave it alone
- Q4_0 quantization of the full model degraded WER from 4.90% to 8.49% on FLEURS English — that's a 73% relative increase, showing how sensitive the encoder is
- Strategy: quantize decoder aggressively, keep encoder at high precision

1.2 Baseline Performance (Your Quality Floor)

The quality metric is **Word Error Rate (WER)** — lower is better. You must maintain ≥80% of baseline performance. Since WER is an error rate, "80% of performance" means your WER can be at most 1.25x the baseline WER (i.e., 25% relative WER increase is the max allowed).

Benchmark	Baseline WER	Max Allowed WER (80% threshold)
FLEURS English	~4.90%	~6.13%
FLEURS French	~varies	~1.25x baseline

FLEURS multilingual avg	~varies	~1.25x baseline

Note: The exact benchmarks haven't been fully disclosed. The eval covers multiple languages (13 supported: English, Chinese, Hindi, Spanish, Arabic, French, Portuguese, Russian, German, Japanese, Korean, Italian, Dutch).

1.3 Inference Engine: vLLM Only

There is NO CHOICE here. Voxtral Realtime's novel architecture is only supported in vLLM. The competition allows vLLM or llama.cpp, but llama.cpp does not support this model architecture.

The official serve command:

```
VLLM_DISABLE_COMPILE_CACHE=1 vllm serve mistralai/Voxtral-Mini-4B-Realtime-2602 \
    --compilation_config '{"cudagraph_mode": "PIECEWISE"}'
```

1.4 Hardware Constraint: NVIDIA L4

The L4 is a mid-range inference GPU:

- 24GB GDDR6 VRAM
- 120W TDP (much lower than A100's 400W)
- Ada Lovelace architecture (SM 8.9)
- Supports FP8, INT8 natively
- No 2:4 structured sparsity acceleration (that's Ampere/A100)

The model at BF16 is ~8GB, fitting comfortably on L4 with room for KV cache. This means the compression challenge is primarily about ENERGY, not about making it fit — it already fits. You need to make it use fewer joules.

PART 2: SET UP YOUR ENVIRONMENT

2.1 System Prerequisites

```
# Verify system
uname -a
free -h            # Should show 32GB
nvidia-smi         # Should show RTX 5080 16GB
```

```
# Install essentials
sudo apt update && sudo apt install -y build-essential git wget curl \
    python3 python3-pip python3-venv ffmpeg libsndfile1
```

2.2 Install vLLM (Nightly Required)

Voxtral Realtime requires the nightly build of vLLM:

```
# Create virtual environment
python3 -m venv ~/voxtral-env
source ~/voxtral-env/bin/activate

# Install vLLM nightly (required for Voxtral support)
pip install vllm --pre --extra-index-url https://wheels.vllm.ai/nightly

# This should automatically install mistral_common >= 1.9.0
# Verify:
python3 -c "import vllm; print(vllm.__version__)"
```

2.3 Install Audio Processing Dependencies

```
pip install soundfile librosa datasets jiwer
pip install mistral_common[audio]

# Install Transformers v5 (needed for native Voxtral support)
pip install transformers>=5.2.0

# For quantization tools
pip install auto-gptq optimum bitsandbytes
pip install GPTQModel  # Has explicit Voxtral support
```

2.4 Login to HuggingFace

```
pip install huggingface-hub
huggingface-cli login
# Paste your access token
```

PART 3: DOWNLOAD THE MODEL

3.1 Download Voxtral Realtime

The model is ~8GB in BF16 — fits easily on your system:

```
mkdir -p ~/models/voxtral-realtime
cd ~/models/voxtral-realtime

huggingface-cli download mistralai/Voxtral-Mini-4B-Realtime-2602 \
    --local-dir . \
    --local-dir-use-symlinks False

# ~8GB download, should take 10-30 minutes
```

3.2 Verify Download and Test Baseline

```
# Check files
ls -la ~/models/voxtral-realtime/

# Quick test: serve the model locally on your RTX 5080
# The BF16 model (~8GB) fits in your 16GB VRAM
VLLM_DISABLE_COMPILE_CACHE=1 vllm serve ~/models/voxtral-realtime \
    --compilation_config '{"cudagraph_mode": "PIECEWISE"}' \
    --max-model-len 16384 \
    --port 8080
```

3.3 Test Transcription

Download a test audio file and verify the model works:

```
# test_voxtral.py
from mistral_common.tokens.tokenizers.audio import Audio
from openai import OpenAI
from huggingface_hub import hf_hub_download
import base64

# Download a sample audio
audio_file = hf_hub_download(
    repo_id="patrickvonplaten/audio_samples",
    filename="bcn_weather.mp3",
    repo_type="dataset"
)

# Read and encode audio
with open(audio_file, "rb") as f:
    audio_b64 = base64.b64encode(f.read()).decode()
```

```
# Connect to vLLM server
client = OpenAI(base_url="http://localhost:8080/v1", api_key="dummy")

# Transcribe
response = client.chat.completions.create(
    model="mistralai/Voxtral-Mini-4B-Realtime-2602",
    messages=[{
        "role": "user",
        "content": [
            {"type": "audio_url", "audio_url": {"url": f"data:audio/mp3;base64,{a
            {"type": "text", "text": "Transcribe this audio."}
        ]
    }],
    max_tokens=500
)
print(response.choices[0].message.content)
```

PART 4: ROUND 1 — CONSERVATIVE COMPRESSION (April 9 → April 27)

Strategy: Component-Aware INT8 Quantization

The key insight: the decoder (3.4B, 85% of params) is compressible, the encoder (0.6B, 15%) is fragile. We quantize them differently.

For Round 1, apply W8A16 quantization to the decoder only, keeping the encoder and adapter at BF16. This is safe and should easily pass WER threshold.

4.1 Approach A: bitsandbytes INT8 (Simplest)

This is the fastest path to a working compressed model:

```
# quantize_voxtral_bnb.py
from transformers import VoxtralRealtimeForConditionalGeneration, AutoProcessor,
import torch

# INT8 quantization config
quantization_config = BitsAndBytesConfig(
    load_in_8bit=True,
    llm_int8_skip_modules=[
        # Keep encoder at full precision
        "whisper_encoder",
```

```
        "audio_encoder",
        # Keep adapter at full precision
        "audio_language_adapter",
        "mm_streams_embeddings",
        # Keep embeddings at full precision
        "embed_tokens",
        "lm_head",
    ]
)

model = VoxtralRealtimeForConditionalGeneration.from_pretrained(
    "mistralai/Voxtral-Mini-4B-Realtime-2602",
    quantization_config=quantization_config,
    device_map="auto",
    torch_dtype=torch.bfloat16,
)

processor = AutoProcessor.from_pretrained("mistralai/Voxtral-Mini-4B-Realtime-260

# Save the quantized model
model.save_pretrained("~/models/voxtral-realtime-int8")
processor.save_pretrained("~/models/voxtral-realtime-int8")

print("INT8 model saved. Size should be ~4.5-5GB (down from ~8GB)")
```

IMPORTANT: bitsandbytes INT8 may not be directly compatible with vLLM serve. If it isn't, use Approach B instead.

4.2 Approach B: GPTQ 8-bit via GPTQModel (Recommended)

GPTQModel has explicit Voxtral support and produces vLLM-compatible output:

```
# quantize_voxtral_gptq.py
from gptqmodel import GPTQModel, QuantizeConfig

# 8-bit GPTQ config — safe for Round 1
quantize_config = QuantizeConfig(
    bits=8,
    group_size=128,
    desc_act=False,  # Faster inference
)

# Load and quantize
model = GPTQModel.from_pretrained(
    "mistralai/Voxtral-Mini-4B-Realtime-2602",
    quantize_config=quantize_config,
```

```
)

# Calibrate with audio transcription data
# GPTQModel needs text calibration data for the decoder
calibration_data = [
    "The weather in Barcelona is sunny with temperatures around 25 degrees.",
    "Welcome to the quarterly earnings call for fiscal year 2025.",
    "Today we'll discuss the latest developments in artificial intelligence.",
    "Die Temperatur in Berlin liegt bei etwa 15 Grad Celsius.",
    "Le président a annoncé de nouvelles mesures économiques.",
    "भारत में मानसून का मौसम जून से सितंबर तक रहता है।",
    "日本の桜の季節は三月から四月にかけてです。",
    "The patient presents with acute respiratory distress syndrome.",
    # Add 200+ more diverse multilingual sentences
    # Include all 13 supported languages
]

model.quantize(calibration_data)

# Save in GPTQ format (compatible with vLLM)
model.save_quantized("~/models/voxtral-realtime-gptq-8bit")
print("GPTQ 8-bit model saved")
```

4.3 Approach C: FP8 Quantization via vLLM (If Supported)

The L4 GPU (Ada Lovelace, SM 8.9) has native FP8 support, making this potentially the most energy-efficient option:

```
# Try serving directly with FP8 quantization
VLLM_DISABLE_COMPILE_CACHE=1 vllm serve mistralai/Voxtral-Mini-4B-Realtime-2602 \
    --compilation_config '{"cudagraph_mode": "PIECEWISE"}' \
    --quantization fp8 \
    --max-model-len 16384 \
    --port 8080
```

If vLLM supports FP8 for Voxtral, this might be the simplest AND most effective approach since L4 has hardware FP8 acceleration.

4.4 Test Your Compressed Model

```
# test_wer.py — Measure Word Error Rate on test audio
import jiwer
from datasets import load_dataset
import requests
import base64
```

```
# Load FLEURS English test set
fleurs = load_dataset("google/fleurs", "en_us", split="test", streaming=True)

predictions = []
references = []

for i, sample in enumerate(fleurs):
    if i >= 50:  # Quick test with 50 samples
        break

    # Get audio
    audio_array = sample["audio"]["array"]
    sr = sample["audio"]["sampling_rate"]

    # Save temp wav
    import soundfile as sf
    sf.write("/tmp/test_audio.wav", audio_array, sr)

    # Transcribe via vLLM server
    with open("/tmp/test_audio.wav", "rb") as f:
        audio_b64 = base64.b64encode(f.read()).decode()

    response = requests.post(
        "http://localhost:8080/v1/chat/completions",
        json={
            "model": "voxtral-realtime",
            "messages": [{
                "role": "user",
                "content": [
                    {"type": "audio_url", "audio_url": {
                        "url": f"data:audio/wav;base64,{audio_b64}"
                    }},
                    {"type": "text", "text": "Transcribe this audio."}
                ]
            }],
            "max_tokens": 500
        }
    )

    pred = response.json()["choices"][0]["message"]["content"]
    ref = sample["transcription"]

    predictions.append(pred)
    references.append(ref)

# Calculate WER
```

```
wer = jiwer.wer(references, predictions)
print(f"Word Error Rate: {wer:.4f} ({wer*100:.2f}%)")
print(f"Threshold (80% of baseline): WER must be < {0.049 * 1.25:.4f}")
```

4.5 Measure Energy Consumption Locally

```
# measure_energy.py — Simulate competition energy measurement
from codecarbon import EmissionsTracker
import time

tracker = EmissionsTracker(
    measure_power_secs=1,
    tracking_mode="process",
    log_level="warning",
)

tracker.start()

# Run your transcription benchmark here
# (transcribe 50-100 audio files through the vLLM server)
import subprocess
subprocess.run(["python3", "test_wer.py"])

emissions = tracker.stop()
print(f"Total energy consumed: {tracker._total_energy.kWh * 3600000:.2f} joules")
print(f"Total CO2 emissions: {emissions:.6f} kg")
```

4.6 Prepare Submission

Step 4.6.1: Create HuggingFace Model Card

```
# Create private repo
huggingface-cli repo create voxtral-realtime-compressed --type model --private

# Upload quantized model
huggingface-cli upload YOUR_USERNAME/voxtral-realtime-compressed \
    ~/models/voxtral-realtime-gptq-8bit/ \
    --private
```

Step 4.6.2: Create vllm_config.yaml

```
# vllm_config.yaml
model: .
dtype: auto
```

```
max_model_len: 131072
gpu_memory_utilization: 0.90
quantization: gptq
compilation_config:
  cudagraph_mode: PIECEWISE
```

If using FP8 instead:

```
# vllm_config.yaml (FP8 variant)
model: .
dtype: auto
max_model_len: 131072
gpu_memory_utilization: 0.90
quantization: fp8
compilation_config:
  cudagraph_mode: PIECEWISE
```

Upload:

```
huggingface-cli upload YOUR_USERNAME/voxtral-realtime-compressed \
    ~/vllm_config.yaml \
    --private
```

Step 4.6.3: Create README.md

```
# Voxtral Realtime 4B Compressed — [Your Team Name]

## Compression Technique

- **Method:** GPTQ 8-bit weight quantization
- **Component-Aware Strategy:** Decoder quantized to INT8, encoder and adapter
  kept at BF16 to preserve audio feature extraction quality
- **Calibration:** Multilingual text data covering all 13 supported languages
- **Embeddings:** Token embeddings and LM head preserved at full precision

## Key Decisions

- Voxtral's audio encoder is extremely sensitive to quantization (Q4 degrades
  WER from 4.9% to 8.5%) — selective quantization of decoder only preserves quali
- GPTQ chosen for vLLM compatibility and Marlin kernel acceleration
- L4 GPU has native FP8/INT8 support, enabling hardware-accelerated inference

## Model Size

- Original: ~8GB (BF16)
```

```
- Compressed: ~5GB (INT8 decoder + BF16 encoder)

## Inference Engine

vLLM (use vllm_config.yaml)

## License

Apache 2.0 (same as original)
```

Step 4.6.4: Submit

Same process as Sarvam: grant organizer access, fill submission form with HuggingFace URI when the form arrives around April 27.

PART 5: ROUND 2 — AGGRESSIVE COMPRESSION (May 20 → June 15)

5.1 GPTQ 4-bit Decoder-Only Quantization

The most impactful Round 2 technique: go to 4-bit on the decoder while protecting the encoder:

```
# quantize_voxtral_w4.py
from gptqmodel import GPTQModel, QuantizeConfig

quantize_config = QuantizeConfig(
    bits=4,
    group_size=128,
    desc_act=True,  # Better quality at 4-bit
)

model = GPTQModel.from_pretrained(
    "mistralai/Voxtral-Mini-4B-Realtime-2602",
    quantize_config=quantize_config,
)

# Use multilingual audio transcription pairs for calibration
# Crucial: include all 13 languages
calibration_data = load_multilingual_calibration()  # Your function

model.quantize(calibration_data)
model.save_quantized("~/models/voxtral-realtime-gptq-4bit")
```

Expected impact:

- Decoder: 3.4B × 4bit = ~1.7GB (down from ~6.8GB at BF16)
- Encoder stays at BF16: ~1.2GB
- Total: ~3GB (from 8GB original = 2.7x compression)
- WER impact on decoder-only quantization: likely small (5-10% relative increase)

5.2 Mixed-Precision: FP8 Encoder + INT4 Decoder

The L4 has native FP8 support. Use FP8 for the encoder (minimal quality loss) and INT4 for the decoder (maximum compression):

```
# This requires custom implementation via LLM-Compressor or manual weight manipul
# Pseudocode for the approach:

from llmcompressor import oneshot
from llmcompressor.modifiers import GPTQModifier, SmoothQuantModifier

# Recipe: SmoothQuant for encoder + GPTQ 4-bit for decoder
recipe = [
    # Smooth and quantize decoder to W4A16
    GPTQModifier(
        scheme="W4A16",
        targets=["language_model.model.layers.*"],  # Decoder only
        ignore=[
            "whisper_encoder.*",           # Don't touch encoder
            "audio_language_adapter.*",    # Don't touch adapter
            "lm_head",                     # Don't touch output
        ],
    ),
]

oneshot(
    model="mistralai/Voxtral-Mini-4B-Realtime-2602",
    recipe=recipe,
    output_dir="./voxtral-mixed-precision",
    dataset="your_calibration_dataset",
    num_calibration_samples=256,
    max_seq_length=2048,
)
```

5.3 Reduce max-model-len (Free Energy Savings)

The default max-model-len is 131072 (~3 hours of audio). The evaluation benchmarks use

short audio clips (typically <30 seconds). Reducing max-model-len saves memory allocation and initialization energy:

```
# vllm_config.yaml — aggressive
model: .
dtype: auto
max_model_len: 16384    # ~20 minutes of audio — more than enough for benchmarks
gpu_memory_utilization: 0.85
quantization: gptq
compilation_config:
  cudagraph_mode: PIECEWISE
```

This reduces pre-computed RoPE frequency tables and KV cache allocation.

5.4 KV Cache Quantization

vLLM supports FP8 KV cache, which is especially effective on L4 (native FP8):

```
# vllm_config.yaml
model: .
dtype: auto
max_model_len: 16384
gpu_memory_utilization: 0.85
quantization: gptq
kv_cache_dtype: fp8_e4m3    # FP8 KV cache — L4 has native support
compilation_config:
  cudagraph_mode: PIECEWISE
```

Voxtral already uses GQA with 8 KV heads (vs 32 Q heads), so the KV cache is already efficient. FP8 halves it further with hardware acceleration on L4.

5.5 AWQ as Alternative to GPTQ

AWQ (Activation-Aware Weight Quantization) may outperform GPTQ for audio models because it preserves activation-sensitive weights:

```
# quantize_voxtral_awq.py
from awq import AutoAWQForCausalLM

model = AutoAWQForCausalLM.from_pretrained(
    "mistralai/Voxtral-Mini-4B-Realtime-2602",
    device_map="auto",
)

# AWQ quantization config
```

```
quant_config = {
    "zero_point": True,
    "q_group_size": 128,
    "w_bit": 4,
}

# Quantize with multilingual calibration
model.quantize(
    tokenizer=None,  # AWQ uses its own calibration approach
    quant_config=quant_config,
    calib_data=calibration_data,
)

model.save_quantized("~/models/voxtral-realtime-awq-4bit")
```

Note: AWQ support for Voxtral in vLLM may need verification. Test before committing.

5.6 Encoder Pruning (Advanced — Round 2 Only)

The encoder has 32 layers with 1280 dimensions. Some middle layers may be redundant. Try structured pruning:

```
# analyze_encoder_layers.py
# Run calibration data through the model and measure layer importance
# via gradient-based or activation-based metrics

import torch
from transformers import VoxtralRealtimeForConditionalGeneration

model = VoxtralRealtimeForConditionalGeneration.from_pretrained(
    "mistralai/Voxtral-Mini-4B-Realtime-2602",
    device_map="auto",
    torch_dtype=torch.bfloat16,
)

# Measure cosine similarity between adjacent encoder layer outputs
# High similarity = redundant layer = safe to remove
similarities = []
hooks = {}
layer_outputs = {}

def make_hook(name):
    def hook_fn(module, input, output):
        layer_outputs[name] = output[0].detach().float().mean(dim=(0,1))
    return hook_fn
```

```
# Register hooks on encoder layers
for i, layer in enumerate(model.whisper_encoder.transformer.layers):
    layer.register_forward_hook(make_hook(f"enc_layer_{i}"))

# Run test audio through model
# ... (process a few audio samples)

# Compare consecutive layer outputs
for i in range(len(layer_outputs) - 1):
    out_a = layer_outputs[f"enc_layer_{i}"]
    out_b = layer_outputs[f"enc_layer_{i+1}"]
    sim = torch.nn.functional.cosine_similarity(out_a, out_b, dim=0)
    similarities.append((i, i+1, sim.item()))
    print(f"Layer {i} → {i+1}: cosine similarity = {sim.item():.4f}")

# Layers with similarity > 0.99 are candidates for removal
```

CAUTION: Encoder pruning is high-risk. The encoder was trained from scratch specifically for causal audio processing. Even small perturbations can significantly degrade WER. Only attempt if you have headroom on the quality threshold after decoder quantization.

5.7 Delay Parameter Optimization

Voxtral supports configurable delay τ (240ms to 2400ms). Lower delay = faster inference = less energy, but worse WER:

- τ = 480ms: WER ~4.9% (matches offline models)
- τ = 240ms: WER ~6-7% (faster but less accurate)
- τ = 2400ms: WER ~4.5% (best quality, most energy)

If the competition evaluates at a specific delay, optimizing this parameter could trade WER headroom for energy savings. Check if τ is configurable via the vllm_config.yaml or if the organizers fix it.

PART 6: QUALITY VALIDATION CHECKLIST

6.1 WER Testing Script

```
# full_wer_test.py
import jiwer
from datasets import load_dataset
import soundfile as sf
```

```
import requests
import base64
import io
import numpy as np

VLLM_URL = "http://localhost:8080/v1/chat/completions"

def transcribe_audio(audio_array, sample_rate):
    """Send audio to vLLM server and get transcription."""
    # Convert to WAV bytes
    buffer = io.BytesIO()
    sf.write(buffer, audio_array, sample_rate, format="WAV")
    audio_b64 = base64.b64encode(buffer.getvalue()).decode()

    response = requests.post(VLLM_URL, json={
        "model": "voxtral-realtime",
        "messages": [{
            "role": "user",
            "content": [
                {"type": "audio_url", "audio_url": {
                    "url": f"data:audio/wav;base64,{audio_b64}"
                }},
                {"type": "text", "text": "Transcribe this audio."}
            ]
        }],
        "max_tokens": 1000
    })
    return response.json()["choices"][0]["message"]["content"]

# Test across multiple languages
languages = {
    "en_us": "English",
    "fr_fr": "French",
    "de_de": "German",
    "es_419": "Spanish",
    "hi_in": "Hindi",
    "ja_jp": "Japanese",
    "zh_cn": "Chinese",
}

for lang_code, lang_name in languages.items():
    try:
        fleurs = load_dataset("google/fleurs", lang_code, split="test", streaming

        predictions = []
        references = []
```

```
        for i, sample in enumerate(fleurs):
            if i >= 20:  # 20 samples per language for quick check
                break

            audio = sample["audio"]["array"]
            sr = sample["audio"]["sampling_rate"]

            pred = transcribe_audio(audio, sr)
            ref = sample["transcription"]

            predictions.append(pred)
            references.append(ref)

        wer = jiwer.wer(references, predictions)
        print(f"{lang_name} ({lang_code}): WER = {wer*100:.2f}%")
    except Exception as e:
        print(f"{lang_name}: Error — {e}")
```

6.2 Quick Sanity Checks

```
# 1. Model loads and serves without errors
VLLM_DISABLE_COMPILE_CACHE=1 vllm serve YOUR_MODEL_PATH \
    --config vllm_config.yaml --port 8080

# 2. Basic English transcription works
curl -X POST http://localhost:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"voxtral","messages":[{"role":"user","content":[{"type":"text","te

# 3. Model info endpoint responds
curl http://localhost:8080/v1/models

# 4. Realtime endpoint exists
curl http://localhost:8080/v1/realtime
```

PART 7: TIMELINE & DAILY PLAN

Week 1: April 9-15 (Setup & Baseline)

- Day 1: Install vLLM nightly, download Voxtral, verify GPU compatibility
- Day 2: Serve BF16 model on RTX 5080, test transcription works

- Day 3: Run WER baseline on FLEURS English + 2-3 other languages
- Day 4: Set up CodeCarbon energy measurement, get baseline energy numbers
- Day 5-6: Try FP8 quantization (simplest — just a vllm serve flag)
- Day 7: If FP8 works, measure WER + energy. If not, start GPTQ path

Week 2: April 16-22 (Quantize & Validate)

- Day 8-9: Apply GPTQ 8-bit quantization (decoder only if possible)
- Day 10-11: Test WER on compressed model across multiple languages
- Day 12: Measure energy consumption with CodeCarbon
- Day 13: Compare: BF16 baseline vs FP8 vs GPTQ-8bit (WER + energy)
- Day 14: Select best approach for Round 1 submission

Week 3: April 23-27 (Submit Round 1)

- Day 15: Create HuggingFace repo, upload model + config
- Day 16: Write README with compression methodology
- Day 17: Final WER validation, submit via form
- Day 18: Buffer day

Week 4-8: May 4 - June 15 (Round 2)

- May 4-19: Experiment with 4-bit decoder, mixed precision, KV cache quantization
- May 20: Round 1 results → analyze leaderboard
- May 20-June 10: Implement winning strategy based on leaderboard data
- June 10-15: Final validation and submission

PART 8: TROUBLESHOOTING

"vLLM fails to load Voxtral"

- Ensure you're using vLLM nightly, not stable release
- Check mistral_common version: `pip show mistral_common` (need >= 1.9.0)

- Try: `pip install -U vllm --pre --extra-index-url https://wheels.vllm.ai/nightly`
- Use `--trust-remote-code` flag if loading from HF

"CUDA error on RTX 5080"

- RTX 5080 is Blackwell (SM 10.0) — ensure CUDA 12.8+ is installed
- vLLM may not fully support SM 10.0 yet. If so, use Colab with L4 for testing
- Try: `--enforce-eager` flag to disable CUDA graph compilation

"FP8 quantization fails"

- RTX 5080 supports FP8 natively (SM 10.0 > SM 8.9 required)
- But vLLM's FP8 kernels may not support Voxtral's architecture yet
- Fall back to GPTQ/AWQ if FP8 fails

"WER too high after quantization"

- Most WER degradation comes from encoder quantization — check if encoder was accidentally quantized
- Try higher bit-width: 8-bit instead of 4-bit
- Use more calibration data in more languages
- Reduce max-model-len to 16384 (reduces precision issues from long RoPE)

"Model produces garbage or all [P] tokens"

- This is the streaming pad issue. Q4 quantization makes the decoder sensitive to speech content in the prefix. The fix is to increase left padding of silence tokens (from 32 to 76 tokens)
- For vLLM, this should be handled automatically, but if you're using a custom pipeline, check the padding configuration

"Energy measurement seems too high"

- Reduce max-model-len (default 131072 wastes energy on initialization)
- Enable CUDA graph compilation: `--compilation_config '{"cudagraph_mode": "PIECEWISE"}'`
- Reduce max-num-batched-tokens for lower latency per request

- Ensure no other GPU processes are running during measurement

---

PART 9: KEY DIFFERENCES FROM SARVAM TRACK

Aspect	Sarvam 30B	Voxtral 4B
Model type	Text LLM (MoE)	Audio ASR (encoder-decoder)
Size	30B (60GB BF16)	4B (8GB BF16)
Eval GPU	A100 80GB	L4 24GB
Inference engine	llama.cpp or vLLM	vLLM ONLY
Quality metric	Accuracy (higher=better)	WER (lower=better)
Compression format	GGUF	GPTQ/AWQ/FP8 (vLLM-compatible)
Fits on your 5080?	No (need cloud GPU)	YES (runs locally in BF16)
Main bottleneck	Memory bandwidth	Model size on L4
Key risk	Indian language degradation	Audio encoder sensitivity
Creative edge	Expert pruning, imatrix	Component-aware mixed precision

---

PART 10: KEY LINKS & RESOURCES

Resource	URL
Voxtral Realtime on HuggingFace	https://huggingface.co/mistralai/Voxtral-Mini-4B-Realtime-2602
Voxtral Realtime Paper	https://arxiv.org/abs/2602.11298
Voxtral.c (architecture reference)	https://github.com/antirez/voxtral.c
Architecture docs (MODEL.md)	https://github.com/antirez/voxtral.c/blob/main/MODEL.md
Q4 GGUF (WER reference)	https://huggingface.co/TrevorJS/voxtral-mini-realtime-gguf

GPTQModel (Voxtral support)	https://github.com/ModelCloud/GPTQModel
vLLM installation	https://docs.vllm.ai/en/latest/getting_started/installation/
FLEURS benchmark	https://huggingface.co/datasets/google/fleurs
jiwer (WER calculation)	https://pypi.org/project/jiwer/
CodeCarbon	https://github.com/mlco2/codecarbon
Resilient AI Challenge	(check your registration email)

APPENDIX A: VOXTRAL WEIGHT LAYOUT

Understanding which weights belong to which component is critical for selective quantization:

```
whisper_encoder.conv_layers.*          → Audio conv stem (PROTECT)
whisper_encoder.transformer.layers.*   → Audio encoder layers 0-31 (PROTECT)
whisper_encoder.transformer.norm.*     → Encoder final norm (PROTECT)

mm_streams_embeddings.embedding_module.audio_language_projection.*
                                       → Adapter MLP (PROTECT)

language_model.model.embed_tokens.*    → Token embeddings (PROTECT)
language_model.model.layers.*          → Decoder layers 0-25 (QUANTIZE THIS)
language_model.model.norm.*            → Decoder final norm (leave)
language_model.lm_head.*              → Output projection (PROTECT)

ada_*                                  → Delay conditioning (leave)
```

The decoder layers (0-25) contain ~3.4B of the 4B total parameters. Quantizing only these gives you ~85% of the compression benefit with minimal impact on audio feature quality.

APPENDIX B: ENERGY ESTIMATION

On L4 (120W TDP), approximate energy per 1-minute audio transcription:

Configuration	Model Size	Approx Energy	Relative
BF16 (baseline)	~8 GB	~15-20 J	1.0x

FP8 (full model)	~4 GB	~8-12 J	~0.6x
W8A16 decoder-only	~5 GB	~10-14 J	~0.7x
W4A16 decoder-only	~3.5 GB	~7-10 J	~0.5x
W4A16 decoder + FP8 encoder	~2.5 GB	~5-8 J	~0.4x

These are rough estimates. Actual energy depends heavily on audio length, batch size, KV cache usage, and vLLM compilation settings.

The key insight: on L4, inference is memory-bandwidth-bound (similar to A100). Every byte of weight not loaded from GDDR6 saves energy. A 4-bit decoder loads 4x fewer bytes per token → roughly 3-4x less decoder energy. But the encoder processes audio features in a compute-bound fashion, so encoder compression saves less energy per byte removed.